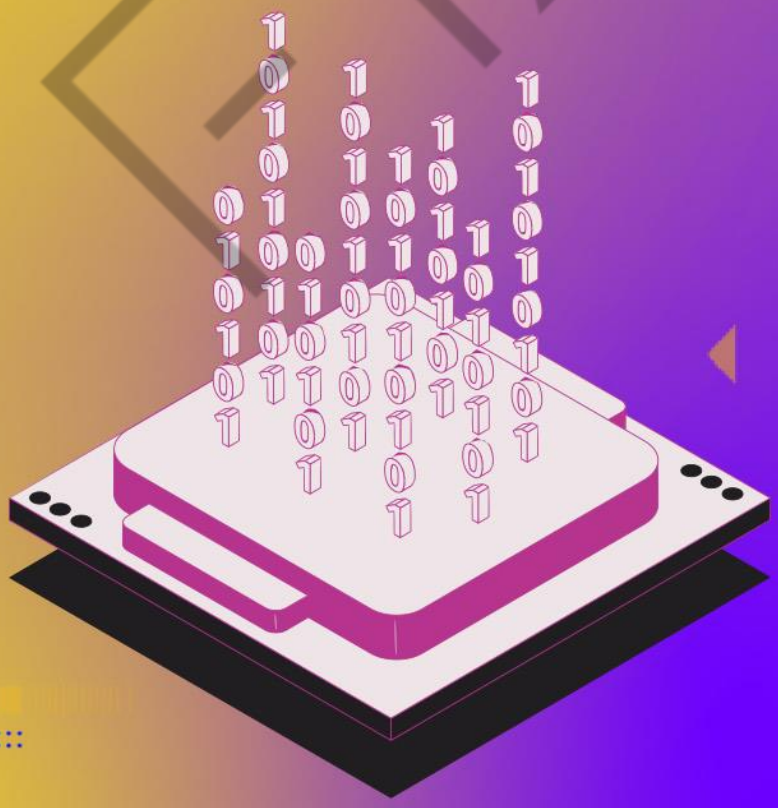
FIAP ON

SAFRA DO CONHECIMENTO
CIBERNÉTICO - PRAGAS VIRTUAIS

FORMATANDO DADOS PARA SISTEMAS DE IA



11

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE TABELAS

Tabela 1 - Resumo dos sistemas numéricos usados em computação.....6
Tabela 2 - Note a coluna Resto e leia de baixo para cima 8
Tabela 3 - Divisão do decimal para o binário 8
Tabela 4 - Posição dos pesos dos bits.....9
Tabela 5 - Posição dos bits 10
Tabela 6 - Detalhando a transformação de decimal para binário 10
Tabela 7 - Detalhando a transformação de decimal para hexa 11
Tabela 8 - Detalhando a transformação de decimal para hexa (2).....12
Tabela 9 - Detalhando a transformação de hexa para decimal 13
Tabela 10 - Detalhando a transformação de bases.....14
Tabela 11 - Transformando de decimal para octal dividindo por 8. Leia de baixo para cima também.....14
Tabela 12 - Transformando de decimal para hexa dividindo por 16. Leia de baixo para cima a coluna Resto.....14
Tabela 13 - Transformando de Binário Decimal aplicando os pesos dos bits 15
Tabela 14 - Transformando de Binário Decimal aplicando os pesos dos bits (2).....15
Tabela 15 - Pesos dos 16 bits 26
Tabela 16 - Conversão de binário para decimal.....27

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Exemplo de uso do uint8_t 18
Código-fonte 2 - Exemplo de uso do uint16_t 19
Código-fonte 3 - Exemplo clássico de uso do int16_t.....20
Código-fonte 4 - Exemplo de uso do uint32_t 20
Código-fonte 5 - Exemplo de uso de interrupção no ESP32 21
Código-fonte 6 - Exemplo de código para medir tempo 25
Código-fonte 7 - Exemplo de promoção da variável.....28
Código-fonte 8 - Exemplo de mudança de tipo de variável 28
Código-fonte 9 - Exemplo de uint16_t.....29
Código-fonte 10 - Exemplo de uso de float 31
Código-fonte 11 - Exemplo de uso de float e multiplicação.....31
Código-fonte 12 - Exemplo de cálculos matemáticos usando math.h 31
Código-fonte 13 - Exemplo de cálculo simples de média.....32
Código-fonte 14 - Exemplo de cálculo do IMC 32
Código-fonte 15 - Exemplo de uso de módulo abs() 33
Código-fonte 16 - Uso de variável booleanas.....34
Código-fonte 17 - Exemplo de uso do bool 35
Código-fonte 18 - Exemplo de como usar botão e acender LED 36
Código-fonte 19 - Exemplo de código para alternar estado do LED usando botão...37
Código-fonte 20 - Exemplo simples de char.....40
Código-fonte 21 - Exemplo simples de String 40
Código-fonte 22 - Exemplo de comparação entre Strings.....40
Código-fonte 23 - Exemplo de como acessar individualmente um caracter dentro da variável.....40
Código-fonte 24 - Exemplo de contagem de palavras.....41
Código-fonte 25 - Parte do código sendo explicado.....41
Código-fonte 26 - Parte do código responsável pela contagem de palavras 42
Código-fonte 27 - Estrutura do laço de repetição usando o for 43
Código-fonte 28 - Parte do código testando a contagem 43
Código-fonte 29 - Testando para encontrar o vazio como o fim.....44
Código-fonte 30 - O return devolve o conteúdo da variável na frente 44
Código-fonte 31 - Nossa velha função setup() de sempre 45
Código-fonte 32 - Exemplo de conversão entre tipos.....46
Código-fonte 33 - Outro exemplo de conversão entre tipos 46
Código-fonte 34 - Exemplo de sensibilidade ao tipo de letra 46
Código-fonte 35 - Indicação de como converter de minúsculo para maiúsculo 46

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 FORMATANDO DADOS PARA SISTEMAS DE IA 5
1.1 Introdução 5
1.1.1 Sistemas numéricos 5
1.1.2 Conversão entre bases 7
2 EXEMPLOS.....14
2.1 Converter o decimal 45 para binário, hexadecimal e octal 14
2.2 Converter o binário 111010 para decimal e hexadecimal 15
3 TIPOS DE DADOS PRIMITIVOS 16
3.1 Números inteiros em programação 16
3.2 Intervalos de valores dos tipos inteiros 17
3.3 Exemplos práticos de uso 18
3.3.1 unit8_t.....18
3.3.2 uint16_t.....19
3.3.3 int16_t.....19
3.3.4 uint32_t com interrupção 20
3.3.5 uint64_t.....24
3.4 Overflow e Underflow 25
3.5 Conversão entre tipos 26
3.6 Comparações e operações aritméticas 28
4 BOAS PRÁTICAS NO USO DE INTEIROS NO ESP32 29
5 PONTO FLUTUANTE 30
5.1 Tipos de dados de ponto flutuante no ESP32 30
5.1.1 Float 30
5.1.2 Double 30
5.2 Declaração e inicialização 31
5.3 Aplicações práticas 31
5.4 Cuidados ao usar ponto flutuante no ESP32.....33
6 BOOLEANA.....34
6.1 Usando variáveis Booleanas no ESP32 34
6.2 Controlando um LED com um botão 35
6.3 Alternando o estado de um LED 37
6.4 Dicas importantes.....38
7 STRING.....39
7.1 Tipos de strings em C/C++ 39
7.1.1 Arrays de Caracteres (C-strings).....39
7.1.2 Objeto `String` (Classe String) 39
7.2 Funções úteis 40
7.3 Dicas para trabalhar com strings no ESP32 45
CONCLUSÃO.....47
REFERÊNCIAS.....48

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 FORMATANDO DADOS PARA SISTEMAS DE IA

1.1 Introdução

A representação de dados é um conceito fundamental na computação, influenciando diretamente a forma como as informações são armazenadas, processadas e transmitidas.

No contexto do ESP32, um microcontrolador famoso e versátil, a compreensão desses conceitos é crucial para o desenvolvimento eficiente de aplicações, especialmente quando combinadas com técnicas de Inteligência Artificial (IA).

Neste capítulo, exploraremos como os dados são representados no ESP32 e como esses conceitos se aplicam ao desenvolvimento de soluções de IA embarcadas.

1.1.1 Sistemas numéricos

É importante entender como os sistemas numéricos influenciam a representação dos dados. Nós, humanos, trabalhamos com o sistema decimal que possui 10 símbolos representados pelo 0, 1, 2, 3, 4, 5, 6, 7, 8 e 9. Qualquer número que usamos em nosso dia a dia é formado por esses algarismos. Contudo, em computação, há outros formatos de numeração, como o binário, octal e o hexadecimal.

A tabela a seguir resume bem como essa representação é feita. Observe que o sistema “humano” decimal está entre o octal e o decimal.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Binário (2 símbolos)	Octal (8 símbolos)	Decimal (10 símbolos)	Hexadecimal (16 símbolos)
0	0	0	0
1	1	1	1
	2	2	2
	3	3	3
	4	4	4
	5	5	5
	6	6	6
	7	7	7
		8	8
		9	9
			A
			B
			C
			D
			E
			F

Tabela 1 - Resumo dos sistemas numéricos usados em computação
Fonte: Elaborado pelo autor (2024)

Existem formas de converter as bases de uma coluna para a outra, e a importância disso é que em algum momento durante a sua jornada no mundo da programação, você vai se deparar com um dado na base hexadecimal, octal ou binário. Por exemplo, considere que você esteja lendo um cartão de RFID para um projeto de liberação de funcionários no uso de uma máquina agrícola, e o resultado do sensor do cartão seja ED BA 03 1F. O que essa sequência de letras e números significa em binário ou decimal? Ou ainda, como comparar esse número consigo mesmo ou com outro número dentro de uma condição simples de “if”, liberando assim o relé que acionará o mecanismo da máquina?

Portanto, trabalhar com diferentes bases numéricas é uma rotina da programação, especialmente ao lidar com endereços de memória, manipulação de

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

bits, protocolos de comunicação e interfaces de hardware. Uma base numérica define o número de dígitos diferentes que um sistema numérico usa para representar números. Repetindo:

- Base 2 (Binária): usa dois dígitos (0 e 1).
- Base 8 (Octal): usa oito dígitos (0 a 7).
- Base 10 (Decimal): usa dez dígitos (0 a 9).
- Base 16 (Hexadecimal): usa dezesseis símbolos (0 a 9 e A a F).

1.1.2 Conversão entre bases

Nesta etapa, aprenderemos as principais pontes de conversão, que serão de decimal para binário, de binário para decimal, de binário para hexadecimal e de hexadecimal para binário. Há outras pontes, como decimal para octal e vice-versa, de octal para binário e vice-versa e de octal para hexa e vice-versa, mas essas seis últimas não vamos abordar agora porque são menos usuais. Teremos foco nas quatro primeiras conversões.

1.1.2.1 Decimal para binário

Para fazer essa conversão, faça uma divisão sucessiva por 2 e depois leia do último quociente para cima, passando pelo resultado do resto. Analise de uma forma mais detalhada:

a. Divida o número decimal por 2;
b. Anote o resto (0 ou 1);
c. Use o quociente para a próxima divisão;
d. Repita até que o quociente seja 0;
e. O número binário é formado pelo o último quociente + restos lidos de baixo para cima. Exemplo: converter o decimal **25** para binário.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Divisão	Quociente	Resto
25 ÷ 2	12	1
12 ÷ 2	6	0
6 ÷ 2	3	0
3 ÷ 2	1	1
1 ÷ 2	0 (fim da linha)	1

Tabela 2 - Note a coluna Resto e leia de baixo para cima
Fonte: Elaborado pelo autor (2024)

Resultado: lendo os restos de baixo para cima, temos **11001**.

Outro exemplo, agora convertendo 73 de decimal para binário. Divida o número decimal por 2 e anote o quociente e o resto em cada etapa:

Etapa	Divisão	Quociente	Resto
1	73 ÷ 2	36	1
2	36 ÷ 2	18	0
3	18 ÷ 2	9	0
4	9 ÷ 2	4	1
5	4 ÷ 2	2	0
6	2 ÷ 2	1	0
7	1 ÷ 2	0	1

Tabela 3 - Divisão do decimal para o binário
Fonte: Elaborado pelo autor (2024)

Anote os restos obtidos em cada divisão:

- Etapa 1: Resto 1
- Etapa 2: Resto 0
- Etapa 3: Resto 0
- Etapa 4: Resto 1
- Etapa 5: Resto 0
- Etapa 6: Resto 0
- Etapa 7: Resto 1

Leia os restos de baixo para cima (da última etapa para a primeira):

1. **Etapa 7**: 1
2. Etapa 6: 0
3. Etapa 5: 0

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4. Etapa 4: 1
5. Etapa 3: 0
6. Etapa 2: 0
7. **Etapa 1**: 1

Os restos lidos na ordem inversa formam o número binário: **1 0 0 1 0 0 1**

1.1.2.2 Binário para decimal

Essa ponte é uma das mais utilizadas. Imagine que você se deparou com um resultado de baixo nível (que significa, nível da máquina) de um erro qualquer em seu código. Como entender esse número? Para converter, teremos somas de potências de 2. Passos:

a. Escreva o número binário e atribua potências de 2 a cada dígito, da direita para a esquerda, começando em 2^0;
b. Multiplique cada dígito binário pela sua potência correspondente;
c. Some todos os resultados.

Exemplo: converter o binário 11001 para decimal.

Posição	Dígito Binário	Potência de 2	Cálculo
4	1	2^4 = 16	1 × 16 = 16
3	1	2^3 = 8	1 × 8 = 8
2	0	2^2 = 4	0 × 4 = 0
1	0	2^1 = 2	0 × 2 = 0
0	1	2^0 = 1	1 × 1 = 1

Tabela 4 - Posição dos pesos dos bits
Fonte: Elaborado pelo autor (2024)

Soma: $16 + 8 + 0 + 0 + 1 = 25$, que, a propósito, é o mesmo número que você fez de decimal para o binário.

Outro exemplo é converter 1001001 de binário para decimal.

a) Anotar os dígitos binários e suas posições. Escreva o número binário e atribua a cada dígito sua posição, começando da direita para a esquerda. A posição corresponde à potência de 2 que será utilizada, iniciando em 0.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Posição (n)	Dígito Binário
6	1
5	0
4	0
3	1
2	0
1	0
0	1

Tabela 5 - Posição dos bits
Fonte: Elaborado pelo autor (2024)

b) Calcular as potências de 2 correspondentes:

- Posição 6: $2^6 = 64$
- Posição 5: $2^5 = 32$
- Posição 4: $2^4 = 16$
- Posição 3: $2^3 = 8$
- Posição 2: $2^2 = 4$
- Posição 1: $2^1 = 2$
- Posição 0: $2^0 = 1$

c) Multiplicar cada dígito pela sua potência de 2:

Posição (n)	Dígito Binário	2^n	Cálculo	Resultado
6	1	64	1 × 64	64
5	0	32	0 × 32	0
4	0	16	0 × 16	0
3	1	8	1 × 8	8
2	0	4	0 × 4	0
1	0	2	0 × 2	0
0	1	1	1 × 1	1

Tabela 6 - Detalhando a transformação de decimal para binário
Fonte: Elaborado pelo autor (2024)

d) Somar todos os resultados:

- 64 (posição 6)
- + 0 (posição 5)
- + 0 (posição 4)
- + 8 (posição 3)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- + 0 (posição 2)
- + 0 (posição 1)
- + 1 (posição 0)

Total: $64 + 0 + 0 + 8 + 0 + 0 + 1 = \mathbf{73}$

1.1.2.3 Decimal para hexadecimal

Para converter qualquer ponte do decimal para uma outra base é necessário dividir pela base que você precisa. Assim como você dividiu por 2 quando quis sair do decimal para o binário, agora você dividirá por 16. Passos:

a) Divida o número decimal por 16;
b) Anote o resto (0 a 15, sendo que os restos de 10 a 15 correspondem de A até F);
c) Use o quociente para a próxima divisão;
d) Repita até que o quociente seja 0;
e) O número hexadecimal é formado pelos restos lidos de baixo para cima.

Exemplo: converter o decimal 754 para hexadecimal.

Divisão	Quociente	Resto
754 ÷ 16	47	2
47 ÷ 16	2	15
2 ÷ 16	0	2

Tabela 7 - Detalhando a transformação de decimal para hexa
Fonte: Elaborado pelo autor (2024)

Restos: 2, 15 (que vira F), 2.

Resultado: lendo os restos de baixo para cima, temos **2F2**. Por que 15 virou F? Porque 15 é uma combinação de 2 algarismos (1 e 5) e só podemos ter 1 algarismo de número e não 2. Logo, qualquer número maior que 9 precisa ser transformado em alguma letra: A ou B ou C ou D ou E ou F, sendo:

- 10 → A
- 11 → B
- 12 → C
- 13 → D

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- 14 → E
- 15 → F

Outro exemplo é converter 999 de decimal para hexadecimal.

a) Divisões sucessivas por 16 e anotando tanto o quociente quanto o resto de cada etapa:

Etapa	Divisão	Quociente	Resto
1	999 ÷ 16	62	7
2	62 ÷ 16	3	14
3	3 ÷ 16	0	3

Tabela 8 - Detalhando a transformação de decimal para hexa (2)
Fonte: Elaborado pelo autor (2024)

b) Converter os restos para dígitos hexadecimais:
   - Resto 7, continua como 7.
   - Resto 14, corresponde à letra E no sistema hexadecimal.
   - Resto 3, continua como 3.

c) Anotar os dígitos na ordem inversa:
   - Etapa 3 = 3
   - Etapa 2 = E
   - Etapa 1 = 7

d) Se formarmos o número hexadecimal, juntando os dígitos na ordem inversa, temos: **3E7**. Portanto, 999 em decimal resulta em 3E7 em hexa.

Na vida real, entender a conversão entre decimal e hexadecimal é crucial para:

- Programação de baixo nível, para manipulação de endereços de memória e dados binários.
- Depuração de código, para análise de dumps de memória e registros.
- Configuração de hardware, para ajuste de registradores e configuração de periféricos.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- Interpretação de códigos de erro, onde muitos sistemas exibem códigos em hexadecimal.

1.1.2.4 Hexadecimal para decimal

Qualquer ponte de uma base computacional (seja 2, 8 ou 16) sendo convertida para a base humana, isto é, base 10, é necessário sempre usar a lógica das potências, somente mudando a base. Entenda os passos:

a) Escreva o número hexadecimal e atribua potências de 16 a cada dígito, da direita para a esquerda, começando em 16^0;
b) Converta cada dígito hexadecimal para decimal;
c) Multiplique cada valor decimal pela sua potência correspondente;
d) Some todos os resultados.

Exemplo: converta o hexadecimal **2F2** para decimal.

Posição	Dígito Hex	Valor Decimal	Potência de 16	Cálculo
2	2	2	16^2 = 256	2 × 256 = 512
1	F	15	16^1 = 16	15 × 16 = 240
0	2	2	16^0 = 1	2 × 1 = 2

Tabela 9 - Detalhando a transformação de hexa para decimal
Fonte: Elaborado pelo autor (2024)

Soma: 512 + 240 + 2 = 754 em decimal.

Agora que você viu as quatro principais conversões entre bases numérica, isto é, de decimal para binário, de binário para decimal, de decimal para hexadecimal e de hexa para decimal, vamos ver onde aplicar isso em programação usando o ESP32.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2 EXEMPLOS

2.1 Converter o decimal 45 para binário, hexadecimal e octal

Decimal para binário:

Divisão	Quociente	Resto
45 ÷ 2	22	1
22 ÷ 2	11	0
11 ÷ 2	5	1
5 ÷ 2	2	1
2 ÷ 2	1	0
1 ÷ 2	0	1

Tabela 10 - Detalhando a transformação de bases
Fonte: Elaborado pelo autor (2024)

Resultado (lendo de baixo para cima): **101101**

Agora, de decimal para octal. Segue um exemplo caso queira entender melhor, embora não nos aprofundamos:

Divisão	Quociente	Resto
45 ÷ 8	5	5
5 ÷ 8	0	5

Tabela 11 - Transformando de decimal para octal dividindo por 8. Leia de baixo para cima também
Fonte: Elaborado pelo autor (2024)

Resultado: 55

Fazendo de decimal para hexadecimal, temos a tabela a seguir:

Divisão	Quociente	Resto
45 ÷ 16	2	13
2 ÷ 16	0	2

Tabela 12 - Transformando de decimal para hexa dividindo por 16. Leia de baixo para cima a coluna Resto
Fonte: Elaborado pelo autor (2024)

Resto 13, que corresponde a D em hexadecimal (lembre-se 10 = A; 11 = B; 12 = C; 13 = D; 14 = E; 15 = F. Não existe 16 = G porque hexadecimal tem 16 símbolos: 0 1 2 3 4 5 6 7 8 9 A B C D E F = 16 símbolos).

Resultado: 2D

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2.2 Converter o binário 111010 para decimal e hexadecimal

Fazendo primeiro o binário para decimal:

Posição	Dígito Binário	Potência de 2	Cálculo
5	1	32	1 × 32 = 32
4	1	16	1 × 16 = 16
3	1	8	1 × 8 = 8
2	0	4	0 × 4 = 0
1	1	2	1 × 2 = 2
0	0	1	0 × 1 = 0

Tabela 13 - Transformando de Binário Decimal aplicando os pesos dos bits
Fonte: Elaborado pelo autor (2024)

Soma: $32 + 16 + 8 + 0 + 2 + 0 = 58$

Agora fazendo de decimal para hexadecimal:

Divisão	Quociente	Resto
58 ÷ 16	3	10
3 ÷ 16	0	3

Tabela 14 - Transformando de Binário Decimal aplicando os pesos dos bits (2)
Fonte: Elaborado pelo autor (2024)

Resto 10, corresponde a “A” em hexadecimal. Resultado = 3A.

A habilidade de converter entre diferentes bases numéricas é importante para qualquer profissional da área da programação que atue nas áreas de IA, computação ou eletrônica. Compreender esses conceitos permite uma melhor interpretação de códigos, depuração de sistemas e desenvolvimento de soluções eficientes. No contexto do ESP32, essa compreensão é ainda mais relevante, pois facilita a interação direta com o hardware e a otimização de recursos.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3 TIPOS DE DADOS PRIMITIVOS

Quando estamos tratando de representação do conhecimento, também temos que tratar da representação dos dados. Em especial, na linguagem C/C++ onde há vários tipos de representação das variáveis. É fundamental entender quais tipos de declaração de variáveis permitem otimizar o seu código, principalmente para economizar posições de memória do microcontrolador.

Inteiros são um dos tipos de dados mais utilizados, representando números sem parte fracionária. No ESP32, e em programação C/C++ em geral, temos uma variedade de tipos inteiros que diferem em tamanho (número de bits) e sinal positivo ou negativo, isto é, com ou sem sinal.

3.1 Números inteiros em programação

São números sem parte decimal, positivos ou negativos (quando com sinal), ou apenas positivos (quando sem sinal). Os inteiros são armazenados em binário, usando uma quantidade fixa de bits (por exemplo, 8, 16, 32 ou 64 bits). No ESP32, os tipos inteiros mais comuns incluem:

- int8_t: inteiro de 8 bits com sinal.
- uint8_t: inteiro de 8 bits sem sinal.
- int16_t: inteiro de 16 bits com sinal.
- uint16_t: inteiro de 16 bits sem sinal.
- int32_t: inteiro de 32 bits com sinal.
- uint32_t: inteiro de 32 bits sem sinal.
- int64_t: inteiro de 64 bits com sinal.
- uint64_t: inteiro de 64 bits sem sinal.

Caso você se pergunte: por que devemos usar tipos específicos? A resposta é: isso garante que uma variável ocupe um número específico de bits em sistemas

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

embarcados com recursos limitados. Além disso, usar tipos com tamanhos definidos evita comportamentos inesperados ao portar código entre diferentes plataformas. E por fim, selecionar o tipo adequado pode otimizar o uso de memória e desempenho do programa.

3.2 Intervalos de valores dos tipos inteiros

Os tipos com sinal podem representar números positivos e negativos. O bit mais significativo (MSB — More Significant Bit) é usado para o sinal:

- int8_t:
  - Tamanho: 8 bits (2^8 = 256 combinações, logo, metade positiva e metade negativa).
  - Intervalo: -128 a 127.
- int16_t:
  - Tamanho: 16 bits (2^16 = 65.536, logo, metade positiva e metade negativa).
  - Intervalo: -32.768 a 32.767.
- int32_t:
  - Tamanho: 32 bits (2^32 = 4.294.967.296, logo, idem).
  - Intervalo: -2.147.483.648 a 2.147.483.647.
- int64_t:
  - Tamanho: 64 bits (2^64 = 18.446.744.073.709.551.616, idem).
  - Intervalo: -9.223.372.036.854.775.808 a 9.223.372.036.854.775.807.

Os tipos sem sinal (unsigned, por isso, o uso da letra “u” na frente do *int*) representam apenas números positivos, permitindo dobrar o valor máximo representável:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- uint8_t:
  - Tamanho: 8 bits.
  - Intervalo: 0 a 255.
- uint16_t:
  - Tamanho: 16 bits.
  - Intervalo: 0 a 65.535.
- uint32_t:
  - Tamanho: 32 bits.
  - Intervalo: 0 a 4.294.967.295.
- uint64_t:
  - Tamanho: 64 bits.
  - Intervalo: 0 a 18.446.744.073.709.551.615.

3.3 Exemplos práticos de uso

3.3.1 unit8_t

Você está programando o ESP32 para controlar o brilho de um LED via PWM (Pulse-Width Modulation). O valor de PWM varia de 0 a 255. O código a seguir é um exemplo de como usar unit8, pois o brilho do LED não pode ser negativo, logo teremos apenas valores sem sinal, ou unsigned, e de até valor 255.

```
     uint8_t brilho = 128; // Valor inicial de brilho (metade do
máximo)

     void setup() {
     .  pinMode(2, OUTPUT); // Define o pino GPIO2 como saída
     }

     void loop() {
       analogWrite(2, brilho); // Ajusta o brilho do LED
       // Código para ajustar 'brilho' conforme necessário
     }
```

Código-fonte 1 - Exemplo de uso do uint8_t
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A declaração **uint8_t brilho = 128** é porque o valor de brilho varia de 0 a 255, não há necessidade de números negativos.

3.3.2 uint16_t

O segundo exemplo será criar um contador que incrementa 1 unidade a cada 1 segundo até um valor máximo de 60.000. Note que os 60.000 serão sempre positivos e menores que o teto de 65.536, pois é o limite máximo que 16 bits pode contar, isto é, 2^16 = 65.536. Logo, adota-se o unit16_t ("u" de "sem sinal" porque não vamos contar números negativos. O sinal só entra quando é número negativo).

```
uint16_t counter = 0;
void setup() {
  // Configuração inicial
}
void loop() {
  delay(1000); // Espera por 1 segundo
  counter++;
  if (counter >= 60000) {
    counter = 0; // Reinicia o contador
  }
  // Utiliza o valor de 'counter' conforme necessário
}
```

Código-fonte 2 - Exemplo de uso do uint16_t
Fonte: Elaborado pelo autor (2024)

3.3.3 int16_t

O terceiro exemplo será um sensor de temperatura que retorna valores negativos e positivos na faixa de -20 a 50 graus Celsius com duas casas decimais de precisão. Caso você deseje evitar o arredondamento das casas decimais para não perder dados e, ao mesmo tempo, trabalhar com números inteiros, será necessário multiplicar as leituras do sensor por 100 para que as duas casas decimais se tornem números inteiros. Por exemplo, se um sensor leu 27,88 então teremos 27,88 * 100 = 2788. Seguindo essa lógica, o sensor que mede de -20 até 50 ºC terá valores entre -20 * 100 até 50 * 100 (de -2000 a 5000).

O int8_t não suportaria essa faixa de números porque seu suporte vai de 0 a 255. Então, analisando a próxima opção, temos o **int16_t**, que vai de -32.768 a 32.767, logo, adotaremos a declaração int16_t.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
     include <DHT.h>

     define pinoDHT 23            //Pino  onde  o  sensor  está
conectado
     define modelo DHT22      //Tipo de sensor DHT22
     DHT dht(pinoDHT, modelo);
     int16_t sensor;

     void setup() {
         Serial.begin(9600);
         dht.begin();
     }

     void loop() {
       sensor  =  dht.readTemperature()*100;  //  considerando
sensor DHT22
       // Utiliza o valor de "sensor" conforme necessário no seu
algoritmo
     }
```

Código-fonte 3 - Exemplo clássico de uso do int16_t
Fonte: Elaborado pelo autor (2024)

3.3.4 uint32_t com interrupção

No quarto exemplo prático, vamos manipular grandes valores com uint32_t, isto é, contar o número de pulsos de um sensor que pode chegar a milhões. E, além disso, apresentaremos um novo conceito chamado Pedido de Interrupção.

```
     uint32_t pulseCount = 0;

     void IRAM_ATTR onPulse() {
       pulseCount++;
     }

     void setup() {
       attachInterrupt(digitalPinToInterrupt(4),       onPulse,
RISING);
       Serial.begin(115200);
     }

     void loop() {
       Serial.println(pulseCount);
       delay(1000);
     }
```

Código-fonte 4 - Exemplo de uso do uint32_t
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

É interessante falar mais deste último código, já que apresentamos um comando inédito, chamado Pedido de Interrupção. Este código demonstra como utilizar interrupções no ESP32 para contar pulsos em um pino digital. É uma aplicação comum em projetos que envolvem contagem de eventos de alta frequência, como a leitura de um sensor de velocidade, contador de voltas ou qualquer dispositivo que gera pulsos digitais.

- Declaração da Variável pulseCount feita por **uint32_t pulseCount = 0;**. Com isso, vimos que uint32_t tem as seguintes características:
  - Tipo de dado: inteiro sem sinal de 32 bits.
  - Intervalo de valores: 0 a 4.294.967.295.
  - Escolhido para permitir contagens altas sem risco de overflow (estouro do valor máximo).
- pulseCount contém o nome da variável que armazenará o número de pulsos contados.
  - Inicializada com zero, representando que nenhum pulso foi detectado no início.

Função de Interrupção onPulse realizada por meio de:

```
void IRAM_ATTR onPulse() {
        pulseCount++;
}
```

Código-fonte 5 - Exemplo de uso de interrupção no ESP32
Fonte: Elaborado pelo autor (2024)

- **void** indica que a função não retorna nenhum valor.
- **IRAM_ATTR** é o atributo específico para o ESP32.
  - **IRAM** refere-se a Internal RAM (memória RAM interna). Informa ao compilador que a função deve ser armazenada na RAM interna, não na Flash. Isso é importante para funções de interrupção, garantindo que sejam executadas rapidamente sem esperas associadas ao acesso à Flash.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **onPulse** é o nome da função de interrupção. Será chamada automaticamente toda vez que a interrupção configurada for acionada.
- **pulseCount++;** serve para incrementar a variável pulseCount em 1. Cada vez que um pulso é detectado, esta função é chamada e o contador é incrementado.
- **void setup()** é a função padrão no Arduino que é executada uma vez no início do programa. Utilizada para configurar pinos, iniciar comunicação serial etc.
- **attachInterrupt()** é a função que associa uma função de interrupção a um pino digital. Seus parâmetros são:
  - **digitalPinToInterrupt(4)** que converte o número do pino digital (4) para o número de interrupção correspondente.
  - **Pino 4** o pino GPIO4 do ESP32 será usado para detectar os pulsos.
  - **onPulse** função a ser chamada quando a interrupção ocorrer.
  - **RISING** define a condição que acionará a interrupção. RISING significa que a interrupção ocorre quando o sinal no pino passa do nível baixo para o nível alto (borda de subida).
- **Serial.begin(115200);** é a função que habilita a comunicação serial entre ESP32 e a porta USP, que será vista no Monitor Serial.
- **void loop()** é a segunda função padrão no Arduino que é executada continuamente após o setup().
- **Serial.println(pulseCount);** serve para imprimir o valor atual de pulseCount no Monitor Serial, seguido por uma nova linha. Permite monitorar quantos pulsos foram contados até o momento.
- **delay(1000);** pausa a execução do loop por 1000 milissegundos (1 segundo). Isso faz com que o valor de pulseCount seja impresso a cada segundo.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Mas afinal, o que é um Pedido de Interrupção? É um mecanismo que permite que o microcontrolador responda a eventos externos imediatamente. Quando uma interrupção ocorre, o fluxo normal do programa é interrompido, e a função associada é executada. As vantagens desse mecanismo são:

- **Eficiência:** não é necessário verificar constantemente (*polling*) se um evento ocorreu.
- **Precisão:** eventos podem ser capturados com precisão temporal.

É importante o uso de IRAM_ATTR pois as funções de interrupção precisam ser rápidas e, por esse motivo, armazenar a função na RAM interna reduz a latência de execução, além de evitar problemas de tempo de acesso associados à memória Flash.

Quanto ao tipo de dados “uint32_t”, ele permite a contagem de um grande número de pulsos sem risco imediato de overflow. O tipo sem sinal é apropriado porque a contagem de pulsos não será negativa.

Nas configurações da interrupção, temos outros modos de detecção:

- **RISING:** a interrupção ocorre na borda de subida (de nível baixo para alto).
- **FALLING:** a interrupção ocorre na borda de descida (de nível alto para baixo).
- **CHANGE:** ocorre em qualquer mudança de estado.
- **HIGH:** a interrupção ocorre enquanto o pino estiver em nível alto. Há modelos de ESP32 que não suportam essa opção. Precisa fazer testes.
- **LOW:** ocorre enquanto o pino estiver em nível baixo.

Como funciona o programa? O programa começa definindo a variável “pulseCount” com valor zero. No “setup()”, inicializamos a comunicação serial e configuramos a interrupção no pino GPIO4. Sempre que ocorre um pulso ascendente no pino GPIO4, a função “onPulse” é chamada. Dentro de “onPulse”, incrementamos “pulseCount” em 1.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

No “loop()”, imprimimos o valor atual de “pulseCount” a cada segundo. Isso permite observar o número total de pulsos detectados desde o início do programa.

Como aplicações práticas, temos os seguintes cenários:

- **Medidores de velocidade:** sua função é contar os pulsos de um sensor Hall em uma roda para calcular a velocidade de um veículo.
- **Fluxo de líquidos:** utilizar um sensor de fluxo que gera pulsos para medir a quantidade de líquido que passou.
- **Encoder rotativo:** utilizado para determinar a posição ou velocidade de um eixo giratório.
- **Contadores de eventos:** contar quantas vezes um evento específico ocorreu, como pessoas passando por um sensor.

É importante mencionar algumas considerações finais sobre o uso de interrupções, como evitar operações complexas na função de interrupção, manter a função simples e rápida, além de também evitar funções como “Serial.print()” dentro de interrupções. Se o programa principal e a função de interrupção modificam a mesma variável, pode haver problemas de acesso simultâneo. Para operações que não são atômicas (não ocorrem em um único ciclo de máquina), devemos desabilitar as interrupções temporariamente ou usar técnicas de sincronização.

No nosso exemplo, incrementos simples em “pulseCount” são operações atômicas em processadores de 32 bits como o ESP32, por isso, não há necessidade de medidas adicionais.

Se “pulseCount” atingir o valor máximo de “uint32_t” (4.294.967.295), ele retornará a zero. A solução é implementar uma lógica para lidar com esse caso, se necessário, dependendo da aplicação.

3.3.5 uint64_t

É possível usar variáveis do tipo uint64_t no ESP32? Sim, embora o ESP32 seja um microcontrolador de 32 bits, ele suporta tipos inteiros de 64 bits, tanto sem sinal (uint64_t) quanto com sinal (int64_t). Você pode declarar e utilizar esses tipos

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

normalmente em seus programas, permitindo manipular números inteiros grandes. Por exemplo, medir o tempo em microssegundos entre eventos, podendo ultrapassar os limites de contagem de 32 bits.

```
uint64_t startTime = 0;
uint64_t endTime = 0;

void setup() {
  Serial.begin(115200);
  startTime = esp_timer_get_time(); // Obtém o tempo em microssegundos

}

void loop() {
  // Código que leva um tempo significativo
  endTime = esp_timer_get_time();
  uint64_t elapsedTime = endTime - startTime;
  Serial.print("Tempo decorrido: ");
  Serial.print(elapsedTime);
  Serial.println(" us");
}
```

Código-fonte 6 - Exemplo de código para medir tempo
Fonte: Elaborado pelo autor (2024)

Observação: fizemos o uso do “uint64_t” pois esp_timer_get_time() retorna um valor de 64 bits em microssegundos.

3.4 Overflow e Underflow

O “overflow” ocorre quando um valor excede o máximo que o tipo pode representar. Por exemplo, “uint8_t x = 255; x++; // x agora é 0, isto é, resetou.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Em relação ao “underflow”, ele ocorre quando um valor cai abaixo do mínimo que o tipo pode representar. Por exemplo, uint8_t x = 0; x--; // x agora assumirá 255.

Dica: é sempre importante verificar se as operações podem causar overflow ou underflow e, se necessário, utilizar tipos maiores.

3.5 Conversão entre tipos

Ao converter um tipo de dado maior para um menor, especialmente de um tipo com maior capacidade de armazenamento para um com menor, podemos enfrentar perda de dados devido ao “overflow”. Isso acontece porque o novo tipo não consegue representar todos os bits do valor original, resultando em truncamento dos bits mais significativos (bits da esquerda).

Por exemplo, vamos converter um `int16_t` (inteiro de 16 bits) para um `uint8_t` (inteiro sem sinal de 8 bits) da seguinte forma:

int16_t x = 300;

uint8_t y = x;

O resultado de “y” será 44. Porém, por que resulta em 44 se antes eram 300? Para responder essa pergunta, vamos aos cálculos de conversão entre bases.

O valor em decimal 300 em binário de 16 bits é: 0000 0001 0010 1100. Quebrando:

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Val	0	0	0	0	0	0	0	1	0	0	1	0	1	1	0	0
Peso	65536	…	…	…	…	…	512	256	128	64	32	16	8	4	2	1

Tabela 15 - Pesos dos 16 bits
Fonte: Elaborado pelo autor (2024)

Como “uint8_t” é um inteiro sem sinal de 8 bits, ele só pode armazenar os 8 bits menos significativos do valor original. Os 8 bits menos significativos de “x” são: 0010 1100, que são os bits 7 6 5 4 3 2 1 0 da direita da tabela.

Para encontrar o valor de “y”, basta calcular o valor desses 8 bits menos significativos (menos significativos porque estão à direita).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Bit (posição)	Valor binário	Peso (2^posição)	Contribuição
7	0	128	0
6	0	64	0
5	1	32	32
4	0	16	0
3	1	8	8
2	1	4	4
1	0	2	0
0	0	1	0

Tabela 16 - Conversão de binário para decimal
Fonte: Elaborado pelo autor (2024)

Ao somar as contribuições, teremos $32 + 8 + 4 = 44$, isto é, y = 44. Portanto, a capacidade de “uint8_t” são valores de 0 a 255 (8 bits sem sinal), o valor original 300 fora do intervalo que “uint8_t” pode representar, apenas os 8 bits menos significativos são considerados e os bits mais significativos são descartados. Uma outra forma de calcular o valor efetivo armazenado em “y” é o resto da divisão de “x” por 256 (2^8). Por exemplo:

$y = x \% 256$

$y = 300 \% 256$

$$y = 44$$

Iremos utilizar analogias para facilitar o entendimento usando um relógio de 12 horas. Imagine que o relógio marca horas de 1 a 12. Se adicionarmos 15 horas ao meio-dia, que horas serão? Para responder essa questão, basta realizar o seguinte cálculo:

$$(12 + 15) \% 12 = 3$$

O relógio mostrará 3 horas. Similarmente, ao ultrapassar o limite do relógio, ele “dá a volta” e começa novamente do início. Retomando o exemplo dos 300, o tipo “uint8_t” pode ser comparado a um “relógio” que conta de 0 a 255. Quando atribuímos um valor maior que 255, ele “dá a volta” e começamos a contar novamente a partir de 0. Exemplificando: 300 – 256 = 44. Ou, matematicamente: y = x % 256.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.6 Comparações e operações aritméticas

Ao comparar ou realizar operações entre tipos diferentes, o compilador promove os tipos para um comum, o que pode causar resultados inesperados. Exemplo problemático:

```
     int8_t a = -1;
     uint8_t b = 1;

     if (a > b) {
       // Este bloco não será executado, pois 'a' é promovido a
int e 'b' a unsigned int
     }
```

Código-fonte 7 - Exemplo de promoção da variável
Fonte: Elaborado pelo autor (2024)

Intuitivamente, poderíamos pensar que ‘a’ vale -1 e ‘b’ vale 1. E, ao comparar: -1 > 1 deveria ser falso. Portanto, o bloco dentro do if não deveria ser executado. Mas o que realmente acontece? Surpreendentemente, dependendo do compilador e do sistema, o bloco pode ser executado, indicando que -1 > 1 é verdadeiro. Isso ocorre devido às regras de promoção de tipos e conversão que o compilador aplica. A solução é converter o unsigned para signed, quando sabemos que o valor unsigned cabe no intervalo do signed. Refazendo o código:

```
     int8_t a = -1;
     int8_t b = 1; // Agora 'b' também é signed

     if (a > b) {
       // Este bloco não será executado, comparação correta entre
signed integers
     }
```

Código-fonte 8 - Exemplo de mudança de tipo de variável
Fonte: Elaborado pelo autor (2024)

Ambas as variáveis são do tipo int8_t. Comparação entre -1 e 1 como signed integers. Resultado: Falso, bloco não é executado.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4 BOAS PRÁTICAS NO USO DE INTEIROS NO ESP32

Sempre leve em consideração a escolha apropriada do tipo, isto é, ao pensar na Memória *versus* intervalo necessário, escolha o menor tipo que acomode o intervalo necessário para economizar memória. Por exemplo, para um contador que vai até 100, use “uint8_t”.

Mantenha consistência nos tipos, ou seja, padronize. Use os tipos inteiros de tamanho fixo (`int8_t`, `uint16_t`, entre outros) para garantir consistência.

Evite tipos ambíguos, pois tipos como `int` e `long` podem variar de tamanho em diferentes plataformas.

Comente o código, indicando o porquê um tipo específico foi escolhido, especialmente se não for óbvio. Por exemplo:

```
    uint16_t sensorValue; // Usamos uint16_t porque o sensor
retorna valores de 0 a 50000
```

Código-fonte 9 - Exemplo de uint16_t
Fonte: Elaborado pelo autor (2024)

E por fim, adote o uso de constantes e macros, como “#define” ou “const” para definir valores máximos e mínimos, facilitando alterações futuras.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5 PONTO FLUTUANTE

O ponto flutuante é uma representação numérica que permite a manipulação de números reais, incluindo frações e números muito grandes ou muito pequenos, que não podem ser representados adequadamente pelos tipos inteiros. Por exemplo, com o ponto flutuante, você pode representar um número em Notação Científica.

No contexto do ESP32, o uso de números em ponto flutuante é essencial em aplicações que exigem cálculos matemáticos complexos, como aquelas encontradas em Machine Learning (ML) e IA.

Números de ponto flutuante são números que têm uma parte inteira e uma parte decimal, como o valor de pi 3.14 ou -0.001.

Por que usar? Nem todos os cálculos podem ser feitos com números inteiros (1, 2, 3…). Quando precisamos representar medidas precisas, como temperatura (23.5 °C) ou probabilidades (0.75), utilizamos números de ponto flutuante.

5.1 Tipos de dados de ponto flutuante no ESP32

Nesta seção, iremos explorar os dois tipos de dados de ponto flutuante no ESP32.

5.1.1 Float

- Utiliza 32 bits de memória.
- Presente para a maioria das aplicações que exigem números decimais.
- Exemplo: “float temperatura = 23.5;”.

5.1.2 Double

- No ESP32, “double” é igual a “float”, mas em outros microcontroladores, double é mais preciso.
- Ambos ocupam 32 bits e têm a mesma precisão (para o ESP32).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- Diferente de computadores, onde “double” é mais preciso.

5.2 Declaração e inicialização

A seguir, temos um exemplo de como declarar o tipo float.

```
float numero1 = 5.75;
float numero2 = -3.14;
```

Código-fonte 10 - Exemplo de uso de float
Fonte: Elaborado pelo autor (2024)

Você pode utilizar números decimais positivos ou negativos. Use o tipo “float” ao declarar suas variáveis. Exemplo de operações básicas:

```
float soma = numero1 + numero2;
float produto = numero1 * numero2;
```

Código-fonte 11 - Exemplo de uso de float e multiplicação
Fonte: Elaborado pelo autor (2024)

Basicamente, você pode realizar adição, subtração, multiplicação e divisão normalmente. Outro exemplo é o uso de funções matemáticas, mas lembre-se de incluir a biblioteca “math.h” para usar funções matemáticas avançadas.

```
#include <math.h>

float raiz = sqrt(numero1);        // Raiz quadrada
float potencia = pow(numero1, 2); // Elevar ao quadrado
float seno = sin(numero2);         // Seno do ângulo
```

Código-fonte 12 - Exemplo de cálculos matemáticos usando math.h
Fonte: Elaborado pelo autor (2024)

5.3 Aplicações práticas

O float ou double são úteis para realizar cálculos precisos, pois os algoritmos de IA geralmente precisam de cálculos precisos com números decimais. Exemplificando, em classificadores, você pode ter que lidar com probabilidades como 0.85 ou 0.15 ou os pesos em Redes Neurais, que são frequentemente números decimais que ajustam como a rede aprende.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Por exemplo, imagine que você está programando o ESP32 para ler as temperaturas de um sensor e quer calcular a média.

```
     float temperatura1 = 22.5;
     float temperatura2 = 23.0;
     float temperatura3 = 22.8;

     float media = (temperatura1 + temperatura2 + temperatura3)
/ 3;

     Serial.print("A temperatura média é: ");
     Serial.println(media);
```

Código-fonte 13 - Exemplo de cálculo simples de média
Fonte: Elaborado pelo autor (2024)

O ESP32 calcula e exibe a temperatura média com casas decimais. Outro exemplo é um classificador simples. Vamos supor que você queira programar um classificador muito simples que decide se uma pessoa está acima do peso baseado no Índice de Massa Corporal (IMC).

```
     float peso = 70.0;        // em quilogramas
     float altura = 1.75;      // em metros

     float imc = peso / (altura * altura);

     Serial.print("O IMC é: ");
     Serial.println(imc);

     if (imc < 18.5) {
       Serial.println("Abaixo do peso");
     } else if (imc < 25.0) {
       Serial.println("Peso normal");
     } else if (imc < 30.0) {
       Serial.println("Sobrepeso");
     } else {
       Serial.println("Obesidade");
     }
```

Código-fonte 14 - Exemplo de cálculo do IMC
Fonte: Elaborado pelo autor (2024)

Como podemos observar, tanto o cálculo do IMC quanto as comparações usam números decimais.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5.4 Cuidados ao usar ponto flutuante no ESP32

Números de ponto flutuante têm precisão limitada e podem não representar exatamente certos números devido ao arredondamento. Por exemplo, 0.1 + 0.2 pode resultar em 0.30000001 em vez de 0.3 exato.

Evite comparar com “==”, pois devido à precisão limitada, comparar números de ponto flutuante com “==” dentro de um “if” pode levar a resultados inesperados.

Uma solução é utilizar uma margem de erro pequena (epsilon). Observe:

```
float a = 0.1 * 3;    // Pode não ser exatamente 0.3
float b = 0.3;
float epsilon = 0.0001;

if (abs(a - b) < epsilon) {
  Serial.println("a e b são aproximadamente iguais");
} else {
  Serial.println("a e b são diferentes");
}
```

Código-fonte 15 - Exemplo de uso de módulo abs()
Fonte: Elaborado pelo autor (2024)

Evite operações desnecessárias, isto é, simplifique os cálculos. Se possível, use operações inteiras ou simplifique fórmulas para reduzir o uso de ponto flutuante.

Prefira “float”, embora no ESP32, “float” e “double” são equivalentes em termos de precisão. Não use “double” esperando mais precisão.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

6 BOOLEANA

As variáveis booleanas são fundamentais na programação, permitindo que você controle o fluxo do seu programa com base em condições verdadeiras ou falsas (True ou False).

Quem foi George Boole? Nascido em 1815, falecido em 1864, foi um matemático e lógico inglês, considerado um dos fundadores da lógica matemática moderna. Ele desenvolveu um sistema de lógica algébrica que é a base da álgebra booleana.

A Lógica de Booleana é uma estrutura algébrica que captura as propriedades das operações lógicas básicas. Opera com valores binários, 0 e 1, que podem representar falso e verdadeiro, respectivamente. Vale lembrar das operações básicas:

- **AND (E):** representado por “&&”, é a operação que resulta em verdadeiro se ambos os operandos forem verdadeiros.
- **OR (OU):** representado por “||”, é a operação que resulta em verdadeiro se pelo menos um dos operandos for verdadeiro.
- **NOT (NÃO):** representado por “!”, é a operação que inverte o valor do operando (verdadeiro torna-se falso, e vice-versa).

Exemplos:

```
bool resultado;

resultado = true && false; // resultado é false
resultado = true || false; // resultado é true
resultado = !true;         // resultado é false
```

Código-fonte 16 - Uso de variável booleanas
Fonte: Elaborado pelo autor (2024)

6.1 Usando variáveis Booleanas no ESP32

Veja esse exemplo de declaração de variável tipo bool:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    bool estadoLED = false; // Inicializa a variável como falsa
    bool sensorAtivado = true; // Inicializa a variável como
verdadeira
```

Código-fonte 17 - Exemplo de uso do bool
Fonte: Elaborado pelo autor (2024)

Em C/C++ (a linguagem usada no Arduino e ESP32), o tipo “bool” é utilizado para variáveis booleanas. Os valores possíveis são `True` (verdadeiro) e `False` (falso).

As variáveis booleanas são frequentemente usadas em estruturas como “if”, “while”, e “for” para controlar o fluxo do programa.

6.2 Controlando um LED com um botão

Nesse exemplo, vamos acender um LED quando um botão for pressionado e apagá-lo quando o botão for solto. Os componentes necessários serão:

- 1 x ESP32
- 1 x LED
- 1 x Resistor de 220 Ω (para o LED)
- 1 x Botão Push-Button

O circuito seria o seguinte: quanto ao LED, terminal Anodo (perna longa) conectado ao pino GPIO 15 através do resistor, e terminal catodo (perna curta) conectado ao GND; quanto ao botão, um terminal conectado ao pino GPIO 2, o outro terminal conectado ao GND. Não esqueça de habilitar o pull-up interno no código quando declarar o pinMode.

Quanto ao código, temos:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
     void setup() {
       Serial.begin(115200);
      // Configura o pino do LED como saída
       pinMode(15, OUTPUT);
       // Configura o pino do botão como entrada com pull-up
interno
       pinMode(2, INPUT_PULLUP);
     }

     void loop() {
       // Lê o estado do botão (LOW quando pressionado devido ao
pull-up)
       bool botaoPressionado = digitalRead(2);

       if (botaoPressionado == LOW) {
         digitalWrite(15, HIGH); // Acende o LED
         Serial.println("Botão pressionado: LED aceso");
       } else {
         digitalWrite(15, LOW); // Apaga o LED
         Serial.println("Botão solto: LED apagado");
       }

       delay(100); // Pequeno atraso para evitar ruído na leitura
     }
```

Código-fonte 18 - Exemplo de como usar botão e acender LED
Fonte: Elaborado pelo autor (2024)

Vejamos a explicação do código:

- **pinMode(2, INPUT_PULLUP);** configura o pino do botão com um resistor de pull-up interno. Isso significa que, quando o botão não está pressionado, o pino lê HIGH, e quando está pressionado, lê LOW. Essa opção é importante para não gerar ruídos na entrada do pino do ESP32, isto é, a entrada sempre terá um nível lógico definido. HIGH quando o botão não estiver pressionado (por isso, PULLUP, que significa, ligado ao nível positivo da bateria do circuito) e LOW quando você pressionar o botão.
- **digitalRead(2);** realiza a leitura do botão, isto é, retorna “true” se o botão estiver liberado, retorna “false” se o botão estiver pressionado, e armazena o resultado na variável booleana “botaoPressionado”.
- **if (botaoPressionado == LOW)** testa se o botão está com LOW, e se essa condição for satisfeita, o LED é aceso.
- **else** se “botaoPressionado” não é LOW, então o LED é apagado.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

6.3 Alternando o estado de um LED

Cada vez que o botão for pressionado, alternar o estado do LED (se estiver apagado, acender; se estiver aceso, apagar).

```
     bool estadoLED = false; // Estado inicial do LED
     bool ultimoEstadoBotao = false; // Para armazenar o último
estado do botão

     void setup() {
       pinMode(15, OUTPUT);
       pinMode(2, INPUT_PULLUP);
       Serial.begin(115200);
     }

     void loop() {
       bool botaoPressionado = digitalRead(2) == LOW;

       // Detecta a transição do botão (borda de descida)
       if (botaoPressionado && !ultimoEstadoBotao) {
         estadoLED = !estadoLED; // Alterna o estado do LED
         digitalWrite(15, estadoLED);
         Serial.print("LED está agora: ");
         Serial.println(estadoLED ? "Aceso" : "Apagado");
       }

       ultimoEstadoBotao  =  botaoPressionado;  //  Atualiza  o
último estado do botão
       delay(50); // Debounce simples
     }
```

Código-fonte 19 - Exemplo de código para alternar estado do LED usando botão
Fonte: Elaborado pelo autor (2024)

Quanto à explicação do código, temos o seguinte:

- **estadoLED** armazena se o LED está aceso (`true`) ou apagado (`false`).
- **ultimoEstadoBotao** guarda o estado anterior do botão para detectar mudanças.
- **estadoLED = !estadoLED;** inverte o valor de “estadoLED”.
- **digitalWrite(15, estadoLED);** atualiza o LED com o novo estado.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- O código verifica se o botão foi pressionado agora (`botaoPressionado == true`) e se estava solto antes (`!ultimoEstadoBotao`). Isso evita múltiplas detecções devido ao ruído ou ao efeito de bounce do botão.

6.4 Dicas importantes

É importante sempre inicializar as variáveis booleanas para evitar comportamentos inesperados. Por exemplo, bool sistemaPronto = false; // Inicializa como falso.

Os nomes que refletem o propósito da variável tornam o código mais legível. Por exemplo, bool sensorAtivado; // Bom. Se fizer assim “bool s;” fica mal documentado, e isto é ruim.

Não é necessário comparar uma variável booleana com `true` ou `false`. Em vez disso, faça: if (sensorAtivado == true) { }. Use: if (sensorAtivado) { }

Atenção aos operadores, como a diferença entre atribuição “=” e comparação “==”. Isso gera o erro comum, assim: if (sensorAtivado = true) { }. O correto é: if (sensorAtivado == true) { }.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

7 STRING

Esse tipo já foi bem explorado anteriormente, e por isso, o objetivo aqui é explorar as ferramentas que manipulam a string. Antes de começarmos, é importante recordar que as strings são sequências de caracteres usados, como letras, números e símbolos, que juntos formam um texto para representar um dado em programação. No contexto de programação básica para IA, strings podem ser usadas para processar e analisar dados textuais. Exemplos:

- "Olá, mundo!"
- "Temperatura atual: 25°C"
- "Comando recebido: LIGAR"

7.1 Tipos de strings em C/C++

No ambiente de programação do ESP32 (baseado em C/C++), existem duas maneiras principais de trabalhar com strings.

7.1.1 Arrays de Caracteres (C-strings)

- Representadas como arrays de `char`.
- Terminadas com um caractere nulo `'\0'` para indicar o fim da string.
- Exemplo: char texto[] = "Olá";

7.1.2 Objeto `String` (Classe String)

- Disponível na biblioteca padrão do Arduino.
- Facilita a manipulação de strings com métodos e operadores.
- Gerenciamento automático de memória.
- Operações simplificadas.
- Exemplo: string texto = "Olá";

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

7.2 Funções úteis

- **strlen(saudacao)** retorna o comprimento da string (número de caracteres, excluindo o `'\0'`).
- **strcpy(destino, origem)** copia uma string para outra.
- **strcat(destino, origem)** concatena (junta) duas strings.

Exemplo 1:

```
char nome[20];
strcpy(nome, "ESP32");
strcat(nome, " é incrível!");
// 'nome' agora contém "ESP32 é incrível!"
```

Código-fonte 20 - Exemplo simples de char
Fonte: Elaborado pelo autor (2024)

Exemplo 2: concatenação.

```
String texto = "Temperatura: ";
int temp = 25;
texto += temp; // "Temperatura: 25"
texto += "°C"; // "Temperatura: 25°C"
```

Código-fonte 21 - Exemplo simples de String
Fonte: Elaborado pelo autor (2024)

Exemplo 3: comparação.

```
if (texto == "Temperatura: 25°C") {
  // As strings são iguais
}
```

Código-fonte 22 - Exemplo de comparação entre Strings
Fonte: Elaborado pelo autor (2024)

Exemplo 4: acesso a caracteres individuais.

```
char primeiroCaractere = texto.charAt(0);
```

Código-fonte 23 - Exemplo de como acessar individualmente um caracter dentro da variável
Fonte: Elaborado pelo autor (2024)

Exemplo 5: contagem de palavras. Imagine que você precisa contar quantas palavras possui uma string. Esse código pode lhe inspirar a resolver.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
String texto = "O ESP32 é ótimo para projetos de IA embarcada.";

int contarPalavras(String frase) {
  int contador = 0;
  bool dentroDaPalavra = false;

  for (int i = 0; i < frase.length(); i++) {
    if (frase.charAt(i) != ' ' && !dentroDaPalavra) {
      dentroDaPalavra = true;
      contador++;
    } else if (frase.charAt(i) == ' ') {
      dentroDaPalavra = false;
    }
  }
  return contador;
}

void setup() {
  Serial.begin(115200);
  int numeroDePalavras = contarPalavras(texto);
  Serial.print("Número de palavras: ");
  Serial.println(numeroDePalavras);
}

void loop() {
  // Não é necessário para este exemplo
}
```

Código-fonte 24 - Exemplo de contagem de palavras
Fonte: Elaborado pelo autor (2024)

Vamos à explicação do código:

```
String texto = "O ESP32 é ótimo para projetos de IA embarcada.";
```

Código-fonte 25 - Parte do código sendo explicado
Fonte: Elaborado pelo autor (2024)

- Variável “texto” é uma string que contém a frase cujo número de palavras queremos contar.
- Conteúdo: “O ESP32 é ótimo para projetos de IA embarcada.”
- Objeto “String”: no Arduino e ESP32, “String” é uma classe que facilita a manipulação de cadeias de caracteres.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- Inicialização: a variável “texto” é inicializada diretamente com a frase desejada.

Função contarPalavras

```
int contarPalavras(String frase) {
  int contador = 0;
  bool dentroDaPalavra = false;

  for (int i = 0; i < frase.length(); i++) {
    if (frase.charAt(i) != ' ' && !dentroDaPalavra) {
      dentroDaPalavra = true;
      contador++;
    } else if (frase.charAt(i) == ' ') {
      dentroDaPalavra = false;
    }
  }
  return contador;
}
```

Código-fonte 26 - Parte do código responsável pela contagem de palavras
Fonte: Elaborado pelo autor (2024)

- Objetivo: contar o número de palavras em uma string passada como parâmetro.
- Parâmetro: recebe uma “String” chamada “frase”.
- Retorno: retorna um inteiro representando o número de palavras encontradas.
- Declaração de Variáveis Locais:
  - **int contador = 0;**
  - Finalidade: armazena a contagem de palavras encontradas.
  - Inicialização: começa em zero, pois nenhuma palavra foi contada ainda.
  - **bool dentroDaPalavra = false;**
  - Finalidade: indica se o algoritmo está atualmente dentro de uma palavra.
  - Inicialização: começa como “false” pois o início da string ainda não foi processado.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Estrutura de controle Loop “for”

```
for (int i = 0; i < frase.length(); i++) {
  // Corpo do loop
}
```

Código-fonte 27 - Estrutura do laço de repetição usando o for
Fonte: Elaborado pelo autor (2024)

- Objetivo: percorrer cada caractere da string `frase`.
- **int i = 0;** inicia o índice `i` em zero.
- **i < frase.length();** continua enquanto `i` for menor que o comprimento da string.
- **i++** incrementa `i` em 1 a cada iteração.

Lógica dentro do Loop

- Caso 1: início de uma Palavra.

```
if (frase.charAt(i) != ' ' && !dentroDaPalavra) {
  dentroDaPalavra = true;
  contador++;
}
```

Código-fonte 28 - Parte do código testando a contagem
Fonte: Elaborado pelo autor (2024)

  - Condição **frase.charAt(i) != ' '** o caractere atual não é um espaço.
  - **!dentroDaPalavra** significa que não estamos atualmente dentro de uma palavra.
  - **dentroDaPalavra = true;** marcamos que agora estamos dentro de uma palavra.
  - **contador++;** incrementamos o contador de palavras, pois encontramos o início de uma nova palavra.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- Caso 2: fim de uma Palavra.

```
else if (frase.charAt(i) == ' ') {
  dentroDaPalavra = false;
}
```

Código-fonte 29 - Testando para encontrar o vazio como o fim
Fonte: Elaborado pelo autor (2024)

  - **frase.charAt(i) == ' '** o caractere atual é um espaço.
  - **dentroDaPalavra = false;** marcamos que saímos de uma palavra.

- Como a lógica funciona?
  - Objetivo geral: detectar transições entre espaços e caracteres não espaciais para contar palavras.
  - Estado dentroDaPalavra: true estamos no meio de uma palavra; false não estamos dentro de uma palavra (pode ser em espaços entre palavras).
- Fluxo de operação:
  - Encontrar o início de uma palavra: se encontramos um caractere que não é espaço e não estávamos dentro de uma palavra, isso significa que uma nova palavra começou.
  - Incrementamos o contador de palavras.
  - Encontrar o fim de uma palavra: se encontramos um espaço, marcamos que não estamos mais dentro de uma palavra.
  - Isso prepara o algoritmo para detectar o início da próxima palavra.

- **Retorno da função:**

  Retorna o número total de palavras encontradas na string.

```
return contador;
```

Código-fonte 30 - O return devolve o conteúdo da variável na frente
Fonte: Elaborado pelo autor (2024)

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **Função setup:**

```
void setup() {
  Serial.begin(115200);
  int numeroDePalavras = contarPalavras(texto);
  Serial.print("Número de palavras: ");
  Serial.println(numeroDePalavras);
}
```

Código-fonte 31 - Nossa velha função setup() de sempre
Fonte: Elaborado pelo autor (2024)

  - Objetivo: configura a velocidade da porta serial para 115200 bauds.
  - Chamada da função “contarPalavras” invoca a função “contarPalavras” passando a string “texto”.
  - O número retornado é armazenado na variável `numeroDePalavras`.
  - **Serial.print()** imprime a string "Número de palavras: " sem uma nova linha.
  - **Serial.println()** imprime o valor de `numeroDePalavras` seguido de uma nova linha.
  - Objetivo: exibir o número de palavras encontradas no Monitor Serial, que será 9.

- **Função loop:**
  - A função `loop` está vazia, pois este programa não precisa executar ações repetidas após o `setup`.

7.3 Dicas para trabalhar com strings no ESP32

O objeto “String” facilita a manipulação de texto, mas pode fragmentar a memória se usado de forma intensiva. Em sistemas com recursos limitados, como o ESP32, isso pode ocasionar problemas. No lugar, busque utilizar arrays de caracteres (`char[]`) para operações simples e, se possível, evite criar muitas instâncias de “String” dinamicamente.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Pode ter conversão entre strings e outros tipos. Por exemplo, string para número:

```
String texto = "123";
int numero = texto.toInt(); // Converte para inteiro
```

Código-fonte 32 - Exemplo de conversão entre tipos
Fonte: Elaborado pelo autor (2024)

E o contrário também, número para string:

```
int valor = 456;
String texto = String(valor); // Converte para String
```

Código-fonte 33 - Outro exemplo de conversão entre tipos
Fonte: Elaborado pelo autor (2024)

Podemos fazer comparação de strings, mas sempre considere a sensibilidade a maiúscula e minúscula:

```
if (comandoRecebido == "LIGAR") {
  // Diferencia "LIGAR" de "ligar" ou "Ligar"
}
```

Código-fonte 34 - Exemplo de sensibilidade ao tipo de letra
Fonte: Elaborado pelo autor (2024)

Uma alternativa é converter a string para maiúsculas ou minúsculas antes da comparação. Por exemplo:

```
comandoRecebido.toUpperCase();    //    Converte    para
maiúsculas
if (comandoRecebido == "LIGAR") {
  // Agora aceita "ligar", "Ligar", etc.
}
```

Código-fonte 35 - Indicação de como converter de minúsculo para maiúsculo
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

CONCLUSÃO

Este capítulo destacou a importância de compreender como os dados são manipulados no contexto da programação de microcontroladores, com foco no ESP32.

A capacidade de entender e aplicar conceitos como sistemas numéricos (binário, octal, decimal e hexadecimal) é crucial para otimizar o uso de memória e melhorar a eficiência das aplicações, especialmente ao lidar com hardware e inteligência artificial embarcada.

Um ponto central é a explicação detalhada das conversões entre diferentes bases numéricas, essenciais para depuração de código, manipulação de endereços de memória e protocolos de comunicação.

Abordamos também os tipos de dados primitivos como inteiros (com e sem sinal) e ponto flutuante, além de exemplos práticos no ESP32, como controle de dispositivos por meio de PWM, contagem de pulsos e leitura de sensores.

Além disso, foram discutidas boas práticas na escolha de tipos de variáveis, sempre considerando a otimização do uso de memória e o comportamento esperado em sistemas embarcados.

Em suma, o capítulo prepara o estudante para implementar soluções robustas e eficientes na sua vida profissional, com aplicações práticas que vão desde controle de hardware até o uso de interrupções e tipos de dados mais complexos como ponto flutuante e strings.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

BABIĆ, Dejan *et al*. **An internet of things system for environmental monitoring based on ESP32 and Blynk**. *In*: 2022 26th International Conference on Information Technology (IT). IEEE, 2022. p. 1-5.

CAPUANO, Francisco Gabriel. **Elementos de eletrônica digital**. Saraiva Educação SA, 2018.

KURNIAWAN, Agus. **Internet of Things Projects with ESP32**: Build exciting and powerful IoT projects using the all-new Espressif ESP32. Packt Publishing Ltd, 2019.

RITCHIE, Dennis M. *et al*. C PROGRAMMING LANGUAGE. **Western Electric engineer**, v. 25, n. 1, p. 14-29, 1981.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com